Firebase in Android Apps

Using Kotlin & Jetpack Compose

Agenda

Part 1: Fundamentals

- Introduction to Firebase
- Setup & Configuration
- Authentication (Auth)
- Cloud Firestore Database

Part 2: Advanced

- Realtime Database
- Cloud Storage
- Cloud Messaging (FCM)
- Architecture & Examples

What is Firebase?

A comprehensive Backend-as-a-Service (BaaS) platform by Google.

Authentication

Identity management and secure sign-in methods.

Firestore

Scalable NoSQL cloud database for storage and sync.

Analytics

Insight into user behavior and app performance.

Setup & Configuration

Integration Steps

- Create project in Firebase Console.
- Register app package name.
- Add google-services.json to app folder.
- Add dependencies using BoM.

Authentication Logic

ViewModel Implementation

Handle authentication state securely using Kotlin Coroutines and StateFlow within a ViewModel.

- Use Firebase.auth instance.
- Expose AuthState (Loading, Success, Error).
- Execute async sign-in operations.

Authentication UI

Jetpack Compose UI

Build reactive login screens that observe the ViewModel state.

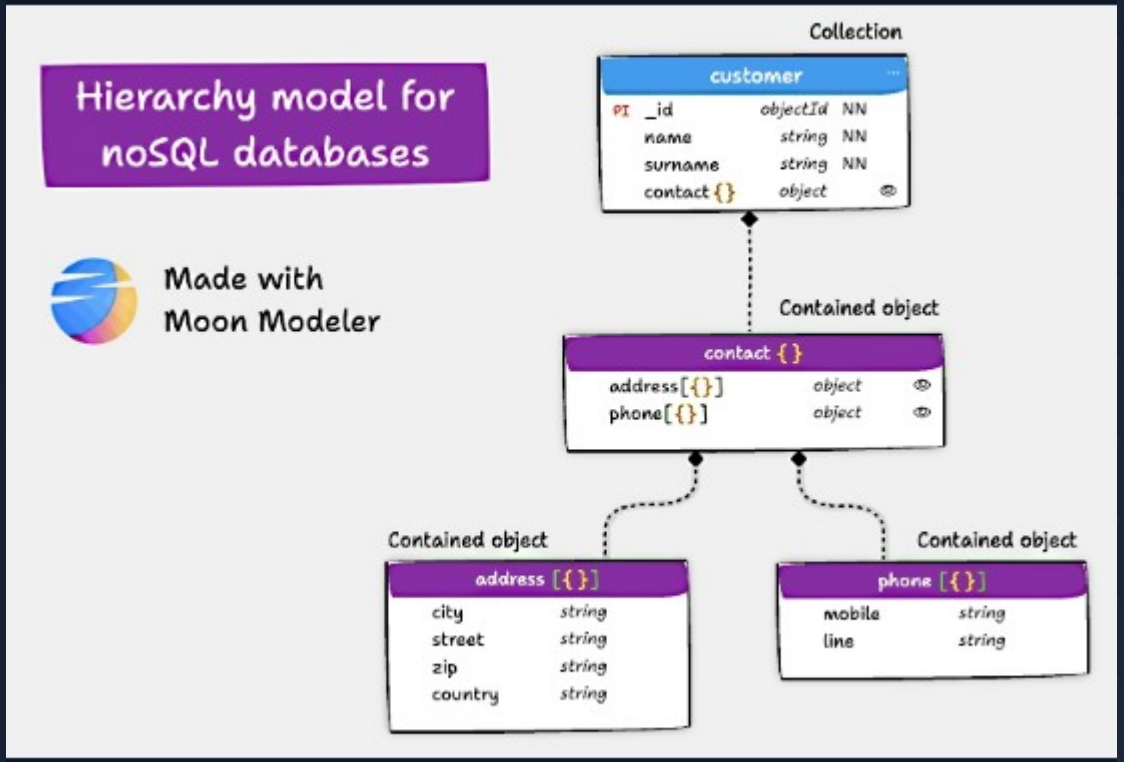
When the AuthState changes (e.g., to Loading), the UI automatically updates to show a progress indicator or error message without manual view manipulation.

Cloud Firestore Structure

Collections & Documents

Firestore is a NoSQL document database. Data is stored in documents, which are organized into collections.

- **Flexible:** No rigid schema.
- **Hierarchical:** Documents can contain subcollections.
- **Real-time:** Listeners update UI instantly when data changes.



Repository Pattern

Clean Architecture

Abstract Firebase calls into a Repository layer. This separates data logic from UI logic and makes the app testable.

Use Kotlin's Result type to handle success and failure states gracefully.

Cloud Storage

Handling Media

Firebase Storage handles robust file uploads and downloads, perfect for user avatars or media sharing.

Using Kotlin Coroutines, you can easily manage asynchronous uploads and track progress percentages to update UI bars in real-time.

Cloud Messaging (FCM)

Service

Extend FirebaseMessagingService to handle incoming messages in the background.

Tokens

Manage FCM registration tokens to target specific devices for notifications.

Channels

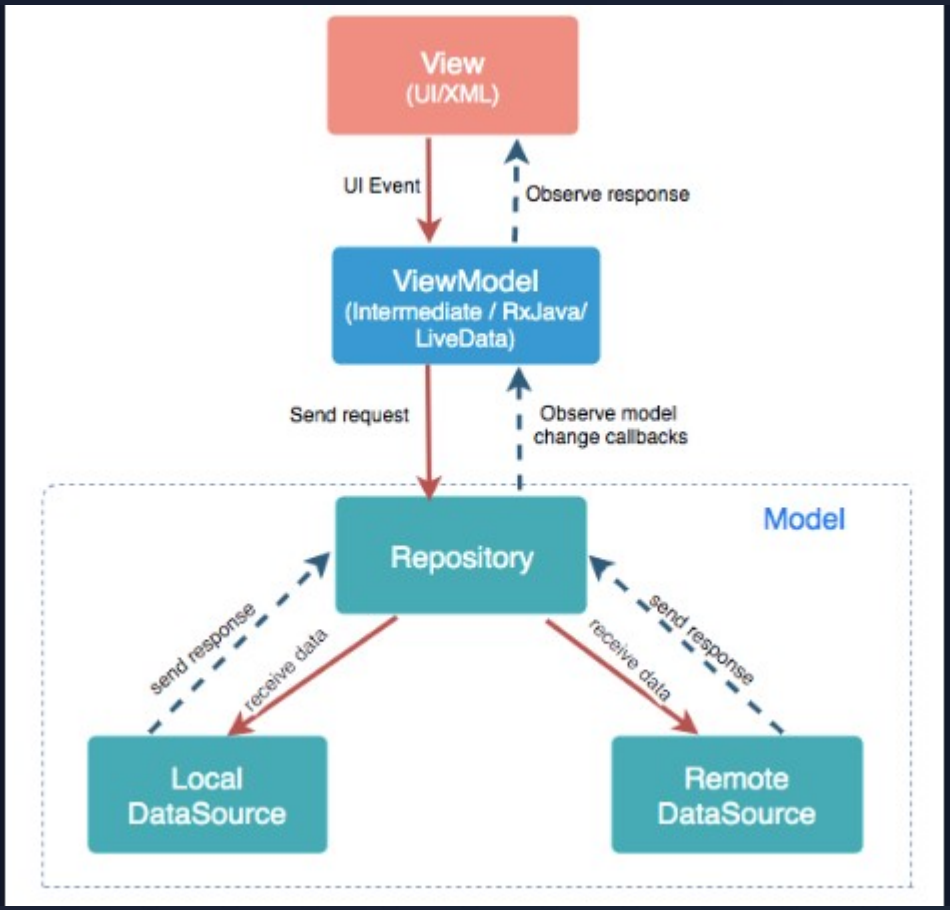
Configure Notification Channels for Android O+ to ensure delivery.

Best Practices: Architecture

MVVM + Repository

Structure your app to be scalable and maintainable.

- **UI Layer:** Composables observe ViewModel state.
- **ViewModel:** Manages UI state and business logic.
- **Repository:** Direct interaction with Firebase SDKs.
- **Security Rules:** Enforce data validation on the backend.

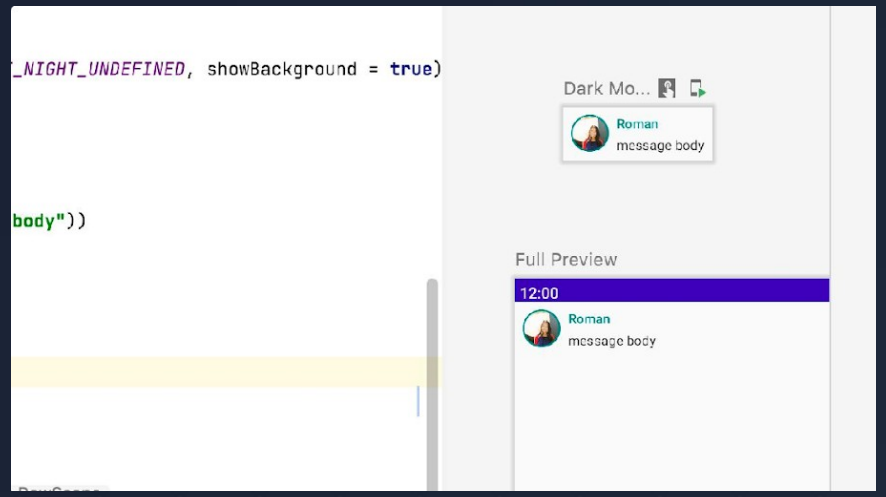


Practical Example: Chat

Real-time Messaging

A chat app demonstrates the power of Firestore's real-time listeners.

Using addSnapshotListener inside a callbackFlow allows the UI to update instantly when a new message document is added to the collection.



Q&A

Thank you!